

《Generative Slate Recommendation with Reinforcement Learning》逐段中文翻译

原文: Romain Deffayet, Thibaut Thonet, Jean-Michel Renders, Maarten de Rijke,
Generative Slate Recommendation with Reinforcement Learning, WSDM 2023.

题目

基于强化学习的生成式推荐列表推荐

作者与单位

Romain Deffayet, Naver Labs Europe (法国 Meylan), 阿姆斯特丹大学 (荷兰阿姆斯特丹)

Thibaut Thonet, Naver Labs Europe (法国 Meylan)

Jean-Michel Renders, Naver Labs Europe (法国 Meylan)

Maarten de Rijke, 阿姆斯特丹大学 (荷兰阿姆斯特丹)

摘要

近期研究已经利用强化学习 (RL) 算法来优化推荐系统中的长期用户参与度, 从而避免用户厌倦和过滤气泡等常见问题。这类方法能够刻画推荐过程的序列性与交互性, 因此为处理长期回报并避免短视行为提供了一种原则性途径。然而, 在列表推荐场景中——即每次交互都要推荐一个物品列表——RL 方法通常难以直接应用, 因为其动作空间具有组合爆炸性质。在该设定下, 一个动作对应于一个推荐列表, 而该列表可能由任意物品组合构成。虽然已有工作提出了若干精心设计的动作分解方法以保证问题可处理, 但这些方法依赖于一些限制性很强、甚至有时并不现实的假设。针对这一问题, 本文提出先将推荐列表编码到一个由变分自编码器学习得到的连续低维潜在空间中, 然后由 RL 智能体在该潜在空间中选择连续动作, 最后再将其解码为相应的推荐列表。通过这种方式, 我们能够: (1) 放宽已有方法所需的假设; (2) 通过对完整列表而非独立单个物品进行建模, 提高动作选择质量, 尤其能够支持多样性。我们在多种模拟环境上的实验表明: 在移除基线方法赖以

成立的限制性假设后，本文所提出的生成式列表建模方法相较于基线具有更优效果。研究表明，基于生成模型的表示学习是实现可泛化的基于 RL 的列表推荐的一条很有前景的方向。

关键词

列表推荐；强化学习；变分自编码器

1 引言

推荐系统在各类在线服务中无处不在，它们通过满足用户已经识别出的兴趣偏好，在个性化服务中发挥关键作用。理想情况下，推荐系统还应当能够提供多样化内容，并帮助用户发现新的兴趣点 [?]。在后一种情形下，推荐系统扮演的是一种主动角色，这意味着推荐会影响用户未来的行为，因此系统对用户产生的影响必须被显式控制。这种影响可能是有害的：如果系统给出过多相似推荐，用户可能会感到厌倦；同时，已有大量研究表明，用户还可能陷入所谓的“过滤气泡”或“回音室”之中。从在线平台或内容提供者的角度看，用户厌倦会导致较差的留存率和转化率，而过滤气泡则带来公平性和伦理层面的问题，平台可能因此被追责。反过来，推荐系统也可能对用户产生积极影响，例如帮助用户对新的、出乎意料的话题产生兴趣，或为可选内容提供更公平的呈现。因此，自然需要在利用（即坚持推荐用户已知感兴趣的内容）与探索（即进一步探测用户兴趣）之间进行平衡，以避免系统总是推荐相似物品，并鼓励那些能够提升未来参与度的推荐。强化学习文献已经提出了一系列模型与算法，旨在通过承认推荐对用户的因果影响来优化长期指标。

本文关注常见的列表推荐 (slate recommendation) 场景，该场景具有其特有的挑战。在每一个交互轮次中，列表推荐系统会从候选集合中给用户推荐一个物品列表，而用户可能与其中零个、一个或多个物品发生交互。因此，用户未必会查看所有被推荐的物品，这会导致观测到的交互带有偏差，同时同一列表内部的物品之间也存在复杂相互作用。更重要的是，动作空间的规模，也就是所有可能列表的数量，使得现成的 RL 方法无法直接使用。因为列表推荐是一个组合优化问题，RL 智能体通过试错来评估所有动作是完全不可行的：即便候选集合中只有 1000 个物品，当列表长度为 10 时，可能的列表数量大约也有 9.6×10^{29} 个。本文试图在一个实际应用场景下解决这一问题，该场景符合许多内容推荐平台的第二阶段排序过程：

(S) 候选集合大约包含一千个物品，并且在每一轮交互中，模型必须从中选择并排序 10 个物品呈现给用户。

因此，本文中所有关于“可处理性”和“可行性”的论述，都应在该场景 (S) 的语境下理解。

为降低组合动作空间的巨大规模，已有研究提出了若干可处理的列表分解方法，但代价是必须引入限制性假设，例如：同一列表中的物品相互独立、已知用户点击模型、可以获得高质量物品嵌入、或者每个列表至多只有一个物品被点击。

与此相对，本文提出首先学习动作（即推荐列表）的连续低维潜在表示，然后在训练阶段让智能体在该潜在空间中执行动作。具体而言，我们通过引入一种基于变分自编码器 (VAE) 的列表生成建模方法 GeMS (Generative Modeling of Slates)，来获得推荐列表的潜在表示。该 VAE 在由旧版推荐系统收集得到的、包含“已观测列表及其点击”的数据集上进行预训练。这类数据集在工业推荐场景中通常是可获得的。因此，我们的方法无需依赖限制性假设；同时，由于我们直接表示完整列表而非单个物品，也使得智能体有机会提升推荐质量。

本文贡献可概括如下：

1. 提出 GeMS，这是一种用于列表推荐 RL 动作表示的新方法：通过在推荐列表及其点击数据上预训练 VAE 来表示动作。与已有方法不同，GeMS 不依赖过强的限制性假设，只要求有日志交互数据。
2. 提供了一套统一术语，用于按照底层假设对已有列表推荐方法进行分类。
3. 在多种模拟环境中表明：一旦移除已有方法所依赖的假设（即更接近实际场景），这些方法的表现会明显下降；而 GeMS 则能够在不依赖限制性假设的前提下恢复高回报策略。
4. 为支持复现，公开了我们的方法、基线方法以及模拟器代码。

2 相关工作

2.1 长期用户参与度

多项研究已经指出，短期收益与长期用户参与度之间往往并不一致，同时传统推荐系统也可能对长期结果产生不利影响。这种短视行为已知会引发用户厌倦并降低用户留存，这无论对用户还是内容提供者都不利。此外，这类行为还会带来“强者愈强”以及加剧封闭认知等问题。已有一些工作试图通过显式最大化多样性，或者通过寻找与长期结果相关的代理指标来缓解这一问题。与之不同，本文直接利用强化学习算法来优化长期指标。

2.2 用于列表推荐的强化学习

已有若干研究讨论了基于强化学习的列表推荐问题，但这些工作的设定各不相同，有些甚至并不适用于本文关注的场景 (S)。Chen 等人与 Bai 等人假设用户点击模型较简单，并进一步假设列表中的物品彼此独立，从而把问题简化为单个物品选择问题，并利用带 SoftMax 策略的 REINFORCE 算法进行求解。Ie 等人假设已知用户点击模型以及物品相关性，因此

可以在计算 Q 值时进行组合优化。Sunehag 等人则在“物品嵌入乘积空间”中采取连续动作，也就是为列表中的每个位置分配一个嵌入，并先选出最近邻物品，再对整列表进行 Q 函数评估。Chen 等人利用最优 Q 函数的性质提出了一种优雅的分解方法，可以自回归地生成最优列表。本文在第 4 节将详细说明这些方法所依赖的假设，但由于其中某一方法计算量过于庞大，我们没有将其纳入实验：在场景 (S) 下，它在每一次训练或推理时都需要大约“列表长度 $\times$ 候选物品数”的神经网络前向传播次数，即约 10000 次。

我们的方法与已有工作不同：我们并不是基于限制性假设对列表进行人工可处理分解，而是用深度生成模型来近似列表生成过程。本文提出的框架只需要一个前提，即能够获取带有推荐列表及其点击的日志数据；在常见工业推荐场景中，这一前提并不苛刻。

2.3 潜在动作表示

在 RL 文献中，学习状态的潜在表示是很常见的，但直接学习动作的潜在表示的研究并不多。Chandak 等人通过监督学习训练动作生成函数，即根据连续两个状态预测其间动作；但在本文场景中，由于真实用户状态不可观测，而相邻观测仅仅是点击信息，信号过弱，不足以反推出导致这些点击的推荐列表。Botteghi 等人以基于模型的方式联合学习潜在状态表示和潜在动作表示，即构建一个状态—动作世界模型。

在我们的场景中，学习世界模型本质上就相当于对“推荐列表与点击”进行潜在建模，同时还要对内部隐藏状态进行条件化。我们在预实验中尝试过类似方法，但额外的条件约束反而会恶化结果，因此本文只报告无条件版本。Zhou 等人的工作与本文较为接近，他们使用 VAE 将动作嵌入到一个可控的潜在空间中，再在其上训练 RL 智能体；但他们研究的是简单原子动作，而不是推荐列表。Jiang 等人与 Liu 等人虽然训练了能够表示推荐列表及其点击的 VAE，但并未进一步研究如何基于该潜在表示训练 RL 智能体。

据作者所知，本文是首个为基于 RL 的推荐任务学习推荐列表潜在表示的工作。

3 方法

3.1 记号与问题定义

我们考虑这样一个列表推荐场景：在一个包含 T 个交互轮次的会话中，用户持续与推荐系统交互。在每个轮次 $t \in \{1, \dots, T\}$ ，系统推荐一个列表

$$a_t = (i_t^1, \dots, i_t^k),$$

其中 $(i_t^j)_{1 \leq j \leq k}$ 来自候选集合 $\mathcal{I}$ ， k 为推荐系统设计者设定的列表长度。用户可以点击列表中的零个、一个或多个物品，系统接收到相应的点击向量

$$c_t = (c_t^1, \dots, c_t^k), \quad c_t^j \in \{0, 1\}.$$

最大化整个会话累计点击数的问题可以建模为一个部分可观测马尔可夫决策过程(POMDP)

$$\mathcal{M}_P = (\mathcal{S}, \mathcal{O}, \mathcal{A}, R, \mathcal{T}, \Omega),$$

其组成如下：

1. 状态集合 $\mathcal{S}$ ，表示用户内在但不可观测的心理状态；
2. 系统可访问的观测集合 $\mathcal{O}$ 。本文中观测为上一次交互的点击，即 $o_t = c_{t-1}$ ，因此观测空间是长度为 k 的二进制向量空间： $\mathcal{O} = \{0, 1\}^k$ ；
3. 动作集合 $\mathcal{A}$ ，即由候选集中的物品构成的所有可能列表，故

$$|\mathcal{A}| = \frac{|\mathcal{I}|!}{(|\mathcal{I}| - k)!};$$

4. 奖励函数 $R : \mathcal{S} \times \mathcal{A} \rightarrow \mathbb{R}$ ，本文设定为

$$R(s_t, a_t) = r_t = \sum_{j=1}^k c_t^j,$$

以反映“最大化累计点击数”的长期目标；

5. 未知的状态转移概率与观测概率，分别为 $\mathcal{T} : \mathcal{S} \times \mathcal{A} \times \mathcal{S} \rightarrow [0, 1]$ 和 $\Omega : \mathcal{S} \times \mathcal{A} \times \mathcal{O} \rightarrow [0, 1]$ ，以及初始状态分布 $S_1 : \mathcal{S} \rightarrow [0, 1]$ 。

由于在 POMDP 中真实用户状态不可观测，通常需要基于可获得的观测构造某种状态代理，并据此训练智能体。生成该状态代理的函数传统上称为“belief encoder”。同时，定义策略 $\pi : \mathcal{S} \times \mathcal{A} \rightarrow [0, 1]$ ，以及轨迹

$$\tau = (o_t, a_t, r_t)_{1 \leq t \leq T}.$$

后文中用 $\tau \sim \pi$ 表示：先由 S_1 采样初始状态 s_1 ，随后递归地按照策略 π 采样动作，最终得到一条轨迹。于是目标可表述为寻找一个最优策略：

$$\pi^* \in \arg \max_{\pi} \mathbb{E}_{\tau \sim \pi} [R(\tau)], \quad R(\tau) = \sum_{t=1}^T r_t.$$

最后，给定状态 s 与动作 a ，定义

$$Q^\pi(s, a) = \mathbb{E}_{\tau \sim \pi, s_1=s, a_1=a} [R(\tau)],$$

以及

$$V^\pi(s) = \mathbb{E}_{a \sim \pi(s)} [Q^\pi(s, a)].$$

3.2 框架概述

在本文提出的框架中，系统与环境（即用户）之间的交互可由如下重复步骤描述：

1. belief encoder 将与用户的历史交互信息总结为一个状态向量；
2. 智能体基于该状态选择一个原型动作（proto-action）；
3. 排序器（本文中由 VAE 模型产生）将该原型动作解码为一个推荐列表，并将其展示给用户。

在本节接下来的内容中，我们首先详细介绍提出的推荐列表生成建模方法 GeMS。GeMS 是一种深度生成模型，它能够为推荐列表及其点击学习一个低维潜在空间；因此这一潜在空间可以作为 RL 智能体方便的原型动作空间，使得我们无需像以往工作那样依赖限制性假设，就能够实现可处理的 RL。随后我们说明如何将 GeMS 作为排序器整合进 RL 框架中，并简要介绍其余 RL 组件。整个两阶段过程如原文第 3 页图 1 所示。

3.3 推荐列表生成建模（GeMS）

为了实例化 GeMS，本文提出在预先收集好的日志交互数据集 $\mathcal{D}$ 上训练一个变分自编码器（VAE），这一过程在原文图 1 左侧给出示意。VAE 的目标是学习数据样本（即推荐列表 a 与点击 c ）和潜在编码（即原型动作 z ）上的联合分布。具体而言，训练一个参数化分布 $p_\theta(a, c, z)$ 来最大化数据的边缘似然：

$$p_\theta(a, c) = \int_z p_\theta(a, c, z) dz.$$

由于该积分通常不可直接计算，因此引入参数化分布 $q_\phi(z | a, c)$ 作为真实后验 $p_\theta(z | a, c)$ 的变分近似，并通过最大化证据下界（ELBO）来训练 VAE：

$$\mathcal{L}_{\theta, \phi}^{\text{ELBO}} = \mathbb{E}_{a, c \sim \mathcal{D}} \left[\mathbb{E}_{z \sim q_\phi(\cdot | a, c)} [\log p_\theta(a, c | z)] - \text{KL}(q_\phi(z | a, c) \| p(z)) \right],$$

其中 $p(z)$ 是潜在空间上的先验分布，KL 表示 Kullback-Leibler 散度，而 z 则通过重参数化技巧从高斯分布中采样得到。

作者指出，最终希望学到的 RL 智能体性能，显然取决于上游 VAE 对推荐列表的重构能力是否足够好。然而，正如 Liu 等人的观察所示，若过分强调列表的精确重构，反而可能限制智能体满足用户兴趣的能力。原因在于，要获得高性能的连续控制策略，潜在空间应当具有足够的平滑性和结构性；如果与先验匹配相关的 KL 项相对过弱，则这种性质可能不足。因此，有必要在重构能力与可控性之间进行平衡。为此，作者在式（1）中的 KL 项前引入超参数 β 作为权重。

此外，为了进一步增强潜在空间的结构性，作者在损失函数中加入了点击重构项：在预训练过程中，具有相似短期结果（即相似点击）的推荐列表会被聚到一起。不过，若过度

偏向点击重构，也可能以牺牲列表重构质量为代价，因此又引入了另一个超参数 λ 来调节这一权衡。作者在第 6.3 节中分析了 β 与 λ 的经验影响。

在实现上，先验 $p(z)$ 被设为标准高斯分布 $\mathcal{N}(0, I)$ 。编码器 $q_\phi(z | a, c)$ 取为具有对角协方差的高斯分布

$$\mathcal{N}(\mu_\phi(a, c), \text{diag}(\sigma_\phi^2(a, c))),$$

其参数由一个多层感知机 (MLP) 输出。该 MLP 的输入为“整个列表上所有可学习物品嵌入与对应点击”的拼接，输出为 $(\mu_\phi(a, c), \log \sigma_\phi(a, c))$ 。对于解码器 $p_\theta(a, c | z)$ ，则由另一个 MLP 以潜在样本 z 为输入，输出每个槽位 j 的重构嵌入 $e_\theta^j(z)$ 以及点击概率 $p_\theta^{j,c}(c^j | z)$ 。然后，作者将重构嵌入 $e_\theta^j(z)$ 与候选集中所有物品的嵌入做点积，从而得到该槽位上各物品概率 $p_\theta^{j,a}(a^j | z)$ 的 logits。对于候选集中的物品嵌入，作者使用编码器内部学习到的当前版本，但通过 stop-gradient 操作阻止梯度反向传播到这些嵌入，以避免潜在的退化解。

综上，VAE 在“重构推荐列表及其对应点击”的任务上进行预训练，即最小化

$$\mathcal{L}_{\theta, \phi}^{\text{GeMS}} = \mathbb{E}_{a, c \sim \mathcal{D}} [\mathcal{L}_{\theta, \phi}^{\text{GeMS}}(a, c)],$$

其中

$$\begin{aligned} \mathcal{L}_{\theta, \phi}^{\text{GeMS}}(a, c) = & \underbrace{\sum_{j=1}^k \log p_\theta^{j,a}(a^j | z_\phi(a, c))}_{\text{列表重构}} + \lambda \underbrace{\sum_{j=1}^k \log p_\theta^{j,c}(c^j | z_\phi(a, c))}_{\text{点击重构}} \\ & + \underbrace{\beta \sum_{i=1}^d (\sigma_{\phi,i}^2 + \mu_{\phi,i}^2 - \log \sigma_{\phi,i} - 1)}_{\text{KL 散度}}. \end{aligned} \quad (1)$$

这里

$$z_\phi(a, c) = \mu_\phi(a, c) + \text{diag}(\sigma_\phi^2(a, c)) \cdot \epsilon, \quad \epsilon \sim \mathcal{N}(0, I),$$

d 为潜在空间维度， β 与 λ 分别控制 KL 项与点击重构项的权重。由于对 $q_\phi(z | a, c)$ 作了高斯假设，且先验为 $\mathcal{N}(0, I)$ ，因此 KL 项具有上述简洁形式。

3.4 RL 智能体与 belief encoder

在完成第 3.3 节所述预训练后，GeMS 的参数被冻结，并将其解码器作为 RL 框架中的排序器。之后，RL 智能体便可以在 VAE 潜在空间中输出原型动作，以最大化折扣回报。对于智能体输出的原型动作 z ，若要生成列表 $(i^1, \dots, i^k)$ ，则对每个槽位 $j \in \{1, \dots, k\}$ ，选择解码器认为最可能的物品：

$$i^j = \arg \max_{i \in \mathcal{I}} p_\theta^{j,a}(i | z).$$

由于本文在 RL 框架中关注的重点是排序器的设计，因此 belief encoder 与智能体本身都采用较为标准的实现：前者使用门控循环单元 (GRU)，输入为每个列表中“物品嵌入与

对应点击”的拼接；后者采用 Soft Actor-Critic (SAC) 算法。作者之所以选择 SAC，是因为它是连续控制领域中成熟、性能强且数据效率较高的 RL 算法；此外，SAC 中包含鼓励探索的熵项，作者在实验中发现，在高度随机的推荐环境中，该机制对于获得高性能十分重要。

4 基线方法及其假设

作者将提出的方法与四类主要基线进行比较，这些基线均来源于已有工作。在本节中，作者说明这些基线及其为使组合型列表推荐问题可处理而引入的用户行为假设。借此，作者能够比较各基线所依赖的假设，并在表 1 中突出本文方法的普适性。需要说明的是，作者只比较这些方法中“列表生成机制”这一与本文主题直接相关的部分，而忽略其他设计细节。

SoftMax

在文献 [3, 8] 中，作者将“列表优化”这一组合问题简化为“物品优化”问题：策略网络输出一个对所有候选物品的 softmax 分布，再从中有放回采样得到列表。这样做需要一个相对温和的假设，即列表的 Q 值可以线性分解为各物品的 Q 值 (DQ)。但更重要的是，它还依赖两个强假设：每个列表至多只有一个物品被点击 (1CL)，以及同一列表内各物品回报彼此独立 (MI)。这两个假设合起来非常强，因为它们实际上意味着列表中某个物品被点击的概率并不依赖于该物品本身。

SlateQ

Ie 等人提出了一种基于模型的方法，其中用户点击行为模型是已知的，而用户动态偏好则由 Q-learning 进行规划与近似。除了前述 DQ 与 1CL 假设外，该方法还要求已知真实相关性与点击模型 (CM)，因此相较于其他方法具有不公平优势。为提高计算效率，本文采用原论文中更快的变体，即 QL-TT-TS。

TopK

尽管据作者所知尚无工作显式提出这一方法，但它是处理列表推荐问题时一种很自然的基线，因此被纳入比较。该方法令智能体在物品嵌入空间中采取连续动作，再根据相似性度量（本文中实际使用点积）选取与该动作最近的 k 个物品作为列表。因此，这一方法假设可以获取历史交互日志 (LD)，以便预训练物品嵌入。实验中作者考虑了两个版本：TopK (MF)，其物品嵌入通过矩阵分解学习得到；TopK (ideal)，其物品嵌入直接使用模拟器内部的真实嵌入。显然，后者具有不公平优势。此外，以这种方式排序还隐含了顺序呈现 (SP) 假设，即用户的真实点击模型是自上而下且逐渐衰减的。

WkNN

Sunehag 等人提出了比 TopK 更细粒度、潜在能力也更强的变体，其思想源于 Wolpertinger。该方法让智能体在“各槽位物品嵌入的乘积空间”中采取动作，即动作维度为 $k \times d$ ，其中 d 为物品嵌入维度。然后，对于列表中的每一个槽位，根据与候选集合中物品嵌入的欧氏距离选出 p 个候选物品，并以贪心方式评估每个候选物品对 Q 值的贡献。除 LD 与 DQ 假设外，WkNN 还依赖两个强假设以保证 Q 函数具有次模性：顺序呈现 (SP) 与“执行最好” (Execution Is Best, EIB)，即短期内看似冒险的推荐从长期看永远不值得。形式上可写为：对于任意策略 π_1, π_2 ，若

$$P(R(s, \pi_1(s)) = 0) \geq P(R(s, \pi_2(s)) = 0),$$

则必有

$$V^{\pi_1}(s) \leq V^{\pi_2}(s).$$

这一假设在某种意义上削弱了长期优化本身的意义。

原文表 1 总结了各基线所依赖的假设。与已有工作相比，本文框架只要求有带推荐列表与对应点击的日志数据 (LD)。作者强调，这一假设在工业推荐系统中通常很容易满足，因此并不构成限制。

除了上述基线，作者还加入了两个参考点：随机策略以及短期预言机 (short-term oracle)。短期预言机能够访问真实的用户嵌入与物品嵌入，因此在每次交互中都能选出相关性概率最高的物品组成列表。于是，它在每一步都能给出即时奖励的上界，但无法处理用户厌倦和推荐影响等长期问题。

5 实验设置

5.1 模拟器

作者设计了一个模拟器，用于观察“放宽各基线所依赖假设”后的影响，并通过多个模拟器变体来验证方法的泛化性。主要设计如下。

物品与用户嵌入 按照场景 (S)，模拟器包含 1000 个物品。作者考虑冷启动设定，即每条新轨迹都会在线生成一个新用户。物品与用户都被随机赋予长度为 20 的嵌入，对应 10 个二维主题： $e = (e^1, \dots, e^{10})$ 。其中每个二维向量 e^t 用于表示主题 t 内部可能存在的子主题。对任意用户或物品 x ，其嵌入生成过程如下：

1. 采样各主题倾向 $w_x^t \sim U(0, 1)$ ，并归一化使得 $\sum_t w_x^t = 1$ ；
2. 采样各主题成分 $\epsilon_x^t \sim \mathcal{N}(0, 0.4 \cdot I_2)$ ，并缩放为 $e_x^t = w_x^t \cdot \min(|\epsilon_x^t|, 1)$ ；
3. 将最终嵌入 $e_x = (e_x^1, \dots, e_x^{10})$ 归一化，使得 $\|e_x\| = 1$ 。

每个物品还关联一个主主题，定义为

$$t(i) = \arg \max_{1 \leq t \leq 10} \|e_i^t\|.$$

为适配不同类型的内容与平台，模拟器中考虑了两类物品嵌入：一类直接由上述过程得到；另一类则对各分量先平方再重新归一化。作者在第 6 节中将前者称为“diffuse embeddings（分散型嵌入）”，后者称为“focused embeddings（聚焦型嵌入）”。

相关性计算 物品 i 对用户 u 的相关性概率是用户嵌入与物品嵌入点积的单调递增函数：

$$\text{rel}(i, u) = \sigma(e_i^\top e_u),$$

其中 σ 为 sigmoid 函数。

厌倦与影响效应 用户嵌入会受到两种机制影响：厌倦（boredom）与影响（influence）。若用户 u 点击了某个物品 i ，则下一轮交互时用户嵌入会更新为

$$e_u \leftarrow \omega e_u + (1 - \omega)e_i,$$

实际实验中取 $\omega = 0.9$ 。此外，如果在用户最近点击的 10 个物品中，有 5 个拥有相同主题 t_b ，则认为用户对该主题产生厌倦，此时将用户在该主题上的嵌入分量置零 5 轮，即 $e_u^{t_b} = 0$ 。这两种机制旨在惩罚短视推荐策略，并鼓励采用长期策略。

点击模型 用户对推荐物品的点击服从位置偏置模型，即点击概率由“物品吸引力”与“位置检查概率”两者相乘得到：

$$P(c \mid i, r) = A_i \times E_r.$$

具体地，对排在位置 r 的物品，定义

$$E_r = \nu \epsilon^r + (1 - \nu) \epsilon^{k+1-r}, \quad \epsilon = 0.85.$$

这意味着点击概率是“自上而下浏览”与“自下而上浏览”两种行为的混合。实验中考虑三种点击模型：

- **TopDown**: $\nu = 1.0$;
- **Mixed**: $\nu = 0.5$;
- **DivPen**: 与 Mixed 相同取 $\nu = 0.5$ ，但若一个列表中有超过 4 个物品来自同一主主题，则将其吸引力 A_i 降低为原来的三分之一，以惩罚缺乏多样性的列表。

在 TopDown 与 Mixed 中，物品吸引力直接取其相关性。综合来看，作者最终在 6 类模拟环境上进行实验，它们由“物品嵌入尖锐程度 (focused 或 diffuse)”与“点击模型 (TopDown、Mixed、DivPen)”两方面共同决定。

5.2 实现与评估细节

实现遵循尽可能标准的 RL 设定，以保证复现性。除 SlateQ 与 SoftMax 外，所有基线及本文方法均与 SAC 配对；SlateQ 使用 Q-learning；而 SoftMax 则与 REINFORCE 搭配，因为它需要离散动作空间，而作者实验中离散版 SAC 的表现更差。所有智能体均采用两层神经网络作为函数逼近器；对于 SlateQ 和 SAC，其 Q 函数均使用目标网络。

对于基线与本文方法共用的超参数，作者首先在若干可能区域内对基线进行网格搜索，再将所选值直接复用到 GeMS 上。所有方法均使用 Adam 优化器，其中 Q 网络学习率设为 0.001，策略网络学习率在适用时设为 0.003；折扣因子 $\gamma = 0.8$ ，Polyak 平均参数 $\tau = 0.002$ 。对于本文方法独有的超参数 (d, β, λ) ，作者在“focused item embeddings + TopDown 环境”上做网格搜索，并选取验证集回报最高的组合，再将该组超参数沿用到所有其他环境。搜索范围为：

$$d \in \{16, 32\}, \quad \beta \in \{0.1, 0.2, 0.5, 1.0, 2.0\}, \quad \lambda \in \{0.0, 0.2, 0.5, 1.0\}.$$

对于需要日志数据假设 (LD) 的方法，作者从一个 ϵ -greedy 预言机策略生成了 10 万条用户轨迹数据集，每条轨迹包含 100 个交互轮次。其中 $\epsilon = 0.5$ ，即每个推荐物品以相等概率来自“均匀随机选择”或“预言机选择”。GeMS 中的 VAE 在该数据集上训练 10 个 epoch，batch size 设为 256，学习率设为 0.001。对于需要预训练物品嵌入的 TopK 与 WkNN，作者在相同数据集上训练一个简单的矩阵分解模型：将“轨迹中的用户”与“该轨迹推荐列表中被点击的每个物品”组成正样本对。

所有实验都在 10 个不同随机种子下重复，比较其平均累计回报。对于 GeMS，随机种子还控制预训练阶段 VAE 的初始化。作者训练智能体共 10 万步；每一步对应一条用户轨迹，而每条轨迹包含 100 个交互轮次。每训练 1000 步，就在 200 条验证用户轨迹上进行一次评估。最终，在测试阶段选取验证回报最高的模型检查点，并在 500 条测试用户轨迹上评测。置信区间采用 Student t 分布计算，显著性检验采用 Welch t 检验，二者都基于 95% 置信水平。

6 结果

作者围绕以下三个研究问题展开实验分析：

1. (RQ1) 在放宽已有方法底层假设的情况下，基于 GeMS 的列表推荐框架与先前方法相比表现如何？

2. (RQ2) 所提出的 GeMS 框架是否能够有效平衡即时回报与未来回报，从而避免用户厌倦？
3. (RQ3) GeMS 中用于平衡的超参数 β 与 λ 会如何影响下游 RL 性能？

6.1 与基线方法的比较 (RQ1)

本节在第 5.1 节给出的六种模拟环境上，对本文方法与基线方法进行比较。

结果概览 原文表 2 给出了在 10 万条用户轨迹训练后，各方法在测试集上的平均回报（即累计点击数）。作者将方法分为两类：一类是 *Disclosed env.*，即利用了环境隐藏信息的方法；另一类是 *Undisclosed env.*，即把环境视为黑箱、因而在实践中可应用的方法。

无论使用哪一种具体环境，首先可以观察到：短期预言机都很容易被多数方法超过。这是因为模拟器会惩罚导致用户厌倦的短视推荐，在这些环境中，要获得更高回报就必须保持多样性。其次，SAC+TopK (ideal) 整体表现非常强，这并不意外，因为它使用了真实物品嵌入这一不公平优势；但这也说明，若在实际方法中引入额外领域知识，性能仍可能进一步提升。相比之下，尽管 SlateQ 可以访问额外环境信息，其表现依然较差，尤其在 DivPen 环境中更为明显。作者推测，这可能源于它采用了近似优化策略，并且依赖“单次只点击一个物品”的限制性假设。

整体比较 总体来看，本文提出的 SAC+GeMS 在全部模拟环境中都优于基线方法。在 6 个测试环境中，SAC+GeMS 的平均结果在 6 个环境里均为最好，其中有 3 个环境对所有其他方法都取得了统计显著更优的结果。SAC+WkNN 的表现很差，作者推测这是因为其动作空间维度更高（本文实验中为 200 维），因此更容易遭遇维数灾难；同时，它还要求候选物品必须在欧氏距离意义下接近目标物品嵌入。SAC+TopK (MF) 有一定竞争力，但其与 SAC+TopK (ideal) 之间的巨大差距说明 TopK 对物品嵌入质量非常敏感。尽管 REINFORCE+SoftMax 所依赖的假设非常强，并且在本文设定下缺少理论保证，但它总体上仍是一个颇具竞争力的基线。不过，作者发现其训练过程不稳定，往往难以收敛；也就是说，尽管其最佳检查点可能有较高回报，但可靠性不足。

不同环境下的比较 对于多数方法而言，TopDown 环境通常是最容易的，不论使用的是 focused 还是 diffuse 物品嵌入。这并不奇怪，因为除随机策略外，几乎所有方法都假设点击模型是自上而下的，或者以自上而下方式采样物品，或者使用来自自上而下日志策略的数据。然而，其他因素也可能主导最终表现，例如 SAC+TopK (MF) 所使用的物品嵌入并不理想。相反，DivPen 对大多数方法来说更困难，因为想取得高回报就必须满足一个更强的额外约束：列表内部必须具备较高多样性。SAC+GeMS 也受到这一动态影响，但它仍能通过生成多样化列表而优于其他方法。最后，与 focused 嵌入相比，使用 diffuse 嵌入似乎不会显著降低 GeMS 的平均回报，但会使其置信区间更大。作者认为，这是因为跨多个主题

的“枢纽型”物品更容易显得有吸引力，但这会削弱更精细的推荐策略，使训练过程更不确定。

6.2 GeMS 通过克服厌倦来提升回报 (RQ2)

作者在引言中指出，基于 RL 的长期优化能够惩罚只推荐高度相关但彼此相似物品的短视行为，因为这种行为会导致用户厌倦。本节验证 SAC+GeMS 是否能够适应性地调整列表选择，以避免厌倦。

回顾第 5.1 节中的模拟环境设定：如果用户最近点击的 10 个物品中有 5 个来自同一主题，则会对该主题产生厌倦；一旦某个主题饱和，用户嵌入中对应维度会被置零，这会降低未来展示给用户的相关物品的吸引力。因此，为了获得更高累计回报，系统必须尽量避免厌倦，即便这意味着要牺牲一部分即时回报。

作者在“focused item embeddings + TopDown”环境中比较了三种方法：

1. 短期预言机 (STO)，始终最大化即时奖励；
2. SAC+GeMS，且 $\gamma = 0.8$ ，即本文主方法；
3. SAC+GeMS，且 $\gamma = 0$ ，即在策略梯度中不显式考虑未来回报。

在该环境中，SAC+GeMS $_{\gamma=0.8}$ 的平均测试回报为 305.3，而 SAC+GeMS $_{\gamma=0}$ 为 194.3，STO 仅为 107.7。由此可见，要取得更高回报，长期优化确实是必需的。乍看之下，SAC+GeMS $_{\gamma=0}$ 的表现仍优于 STO 似乎有些反直觉，但作者解释说，其训练目标虽然只优化平均即时回报，却也会隐式促使模型避免未来出现极低回报；不过，当显式考虑推荐对用户的因果影响（即设置 $\gamma = 0.8$ ）时，系统的决策能力会进一步提升。

原文图 2 画出了三种方法所推荐物品的相关性分数分布，这里的分数定义为模拟器内部用户嵌入与物品嵌入的点积。虚线表示达到 0.5 相关性概率所需的阈值 0.28，因此虚线左侧的物品点击概率较低，右侧则较高。图中颜色表示在推荐该物品时有多少个主题已经饱和：可以看到，当用户对至少一个主题产生厌倦时，物品得分会明显下降，因此吸引力也会减弱。

当没有任何主题饱和时（即黄色分布），STO 推荐的物品分数极高，通常高于阈值，甚至可达 0.45；因此它能获得很高的即时回报。但正因如此，它也引发了大量厌倦（图中橙色区域较大），从整体上降低了期望得分（红色实线），最终反而带来更少点击。相比之下，SAC+GeMS $_{\gamma=0.8}$ 会牺牲一部分即时回报（黄色分布整体略向左移），但几乎不会引发厌倦（橙色区域很小）。也就是说，它通过相关性多样性之间权衡，既保持了较好的即时收益，又有效限制了厌倦，因此平均得分更高。SAC+GeMS $_{\gamma=0}$ 则表现出介于两者之间的行为：它推荐的物品相关性有较大波动，但仍会产生相当明显的厌倦，其橙色区域明显大于 $\gamma = 0.8$ 的情形。

6.3 平衡超参数 β 与 λ (RQ3)

在第 3.3 节中，作者提出： β 与 λ 的选择将带来不同权衡，并最终影响 SAC+GeMS 的下游性能。回顾其含义， β 控制“精确重构”与“潜在空间平滑性/结构性（即可控性）”之间的权衡，而 λ 则控制“点击重构”相对于“列表重构”的权重。作者在“focused item embeddings + TopDown 环境”上统计了不同超参数取值对应的最佳验证集回报，结果如原文图 3 所示。

图 3(a) 表明，在 β 的选择上，确实存在一个“甜点区间”。这一结果验证了第 3.3 节中的直觉以及 Liu 等人的观察：为了在下游 RL 任务上获得高性能， β 必须被恰当地平衡。具体而言，不论潜在维度取 16 还是 32，作者都发现 $\beta = 1.0$ 能够带来最高回报。

图 3(b) 显示，“列表重构”与“点击重构”之间的权衡对下游性能的影响虽然没有 β 那么显著，但仍然是可观察的。这也说明作者在损失函数中加入点击重构项（式 (1)）是合理的，尽管在 RL 训练阶段 GeMS 解码器输出的点击并不会被直接使用。换言之，将点击与推荐列表联合建模可以提升 SAC+GeMS 的最终表现，但必须通过 λ 对点击重构项进行适当加权，否则会影响列表重构本身的效果。

7 结论

本文提出了 GeMS，一种基于变分自编码器的推荐列表表示学习方法，用于强化学习驱动的专栏推荐。与现有方法相比，该方法的显著优势在于：它具有更高灵活性，能够对完整推荐列表进行建模，同时只依赖较轻量的假设。

研究发现与更广泛影响 在多种环境上的实验表明，GeMS 在实际设定下相较于现有列表表示方法具有更优性能。进一步的经验分析还显示，它确实能够平衡即时回报与未来回报，而且 β 与 λ 所引入的权衡会显著影响 RL 的下游性能，这说明恰当地设置这些超参数十分关键。总体而言，本文结果表明：对于推荐列表这类复杂动作，采用生成模型进行表示是一条很有前景的研究方向。

局限性 本文的模拟实验展示了 GeMS 作为 RL 框架中列表表示方法的有效性。然而，众所周知，在线训练 RL 智能体成本高且风险大，因此在实际中，智能体往往需要离线训练，也就是直接利用日志数据进行学习。本文并未处理离线 RL 的特有挑战，因为作者希望先把“推荐列表表示”这一因素对下游性能的贡献单独剥离出来进行研究。

未来工作 未来，作者计划进一步研究如何在离线设定中利用生成模型，如何将该思想拓展到不同场景或更复杂动作上，以及如何借助领域知识和用户模型改进结构化动作表示所使用的模型架构。

致谢

本研究部分由 Hybrid Intelligence Center 资助。该中心是荷兰教育、文化与科学部通过荷兰科学研究组织资助的一项为期十年的计划。文中所有内容仅代表作者观点，不代表其雇主或资助方的立场。

参考文献说明

原文第 8-9 页给出了 36 篇参考文献。由于用户需求是“逐段翻译文章正文并导出为 L^AT_EX 与 PDF”，本译稿保留正文完整中文翻译，不再逐条翻译参考文献标题；若需要，我可以继续单独生成一版“参考文献题目中文对照版”。